

README

WebSocket Translation Monitoring System

A comprehensive real-time monitoring system for translation workflows with WebSocket support, SSH worker integration, and interactive web dashboard.

Quick Start

1. Start Monitoring Server

```
go run ./cmd/monitor-server
```

2. Open Monitoring Dashboard

- **Basic Dashboard:** <http://localhost:8090/monitor>
- **Enhanced Dashboard:** [enhanced-monitor.html](#)

3. Run Translation with Monitoring

Basic demo with WebSocket monitoring

```
go run demo-translation-with-monitoring-fixed.go
```

Comprehensive demo with multiple strategies

```
go run demo-comprehensive-monitoring.go
```

SSH worker demo (if configured)

```
go run demo-ssh-worker-with-monitoring.go
```

Real LLM translation (requires API key)

```
export OPENAI_API_KEY=your_key_here
```

```
go run demo-real-llm-with-monitoring.go
```

4. Use Interactive Demo Script

```
chmod +x scripts/run_monitoring_demo.sh
```

```
./scripts/run_monitoring_demo.sh
```

What This System Provides

Real-Time Monitoring

- **Live Progress Updates:** WebSocket-based real-time progress tracking
- **Event Streaming:** Comprehensive event emission and handling
- **Session Management:** Multiple simultaneous translation sessions
- **Error Tracking:** Immediate error detection and reporting

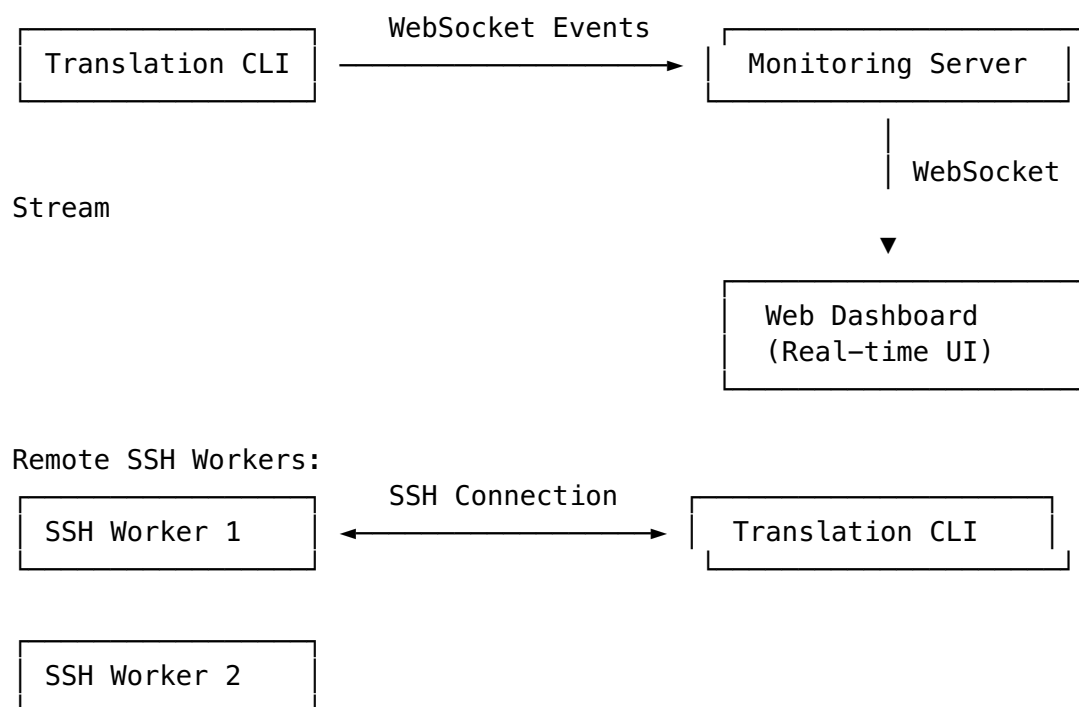
Worker Integration

- **SSH Worker Support:** Remote worker management and monitoring
- **LLM Integration:** Real translation with OpenAI, Anthropic, DeepSeek, etc.
- **Multiple Strategies:** Demo, mock-LLM, and SSH simulation modes
- **Performance Tracking:** Worker capacity and performance metrics

Interactive Dashboard

- **Progress Visualization:** Real-time progress bars and charts
- **Event Logging:** Detailed event history and filtering
- **Session History:** Past translation session records
- **Worker Information:** SSH worker status and details

Architecture



Project Structure

Core Components

- `cmd/monitor-server/` - WebSocket monitoring server
- `pkg/websocket/` - WebSocket hub and connection management
- `pkg/events/` - Event system architecture
- `pkg/sshworker/` - SSH worker management

Demo Applications

- `demo-translation-with-monitoring-fixed.go` - Basic WebSocket monitoring demo
- `demo-comprehensive-monitoring.go` - Multi-strategy comprehensive demo
- `demo-ssh-worker-with-monitoring.go` - SSH worker integration demo
- `demo-real-llm-with-monitoring.go` - Real LLM translation demo

Dashboard & UI

- `monitor.html` - Basic monitoring dashboard
- `enhanced-monitor.html` - Advanced dashboard with SSH worker support

Configuration

- `config.json` - Main application configuration
- `internal/working/config.distributed.json` - SSH worker configuration
- `internal/working/config/*.json` - Various LLM provider configs

Documentation

- `docs/WebSocket_Monitoring_Guide.md` - Complete technical documentation
- `docs/User_Guide.md` - Step-by-step user instructions
- `docs/Troubleshooting_Guide.md` - Common issues and solutions
- `tests/websocket_monitoring_test.go` - Comprehensive test suite

Scripts

- `scripts/run_monitoring_demo.sh` - Interactive demo script

Features

WebSocket Monitoring

- **Real-time Events:** Translation progress, errors, completion events
- **Multi-client Support:** Multiple dashboard connections simultaneously
- **Session-based Tracking:** Unique session IDs for each translation
- **Automatic Reconnection:** Client reconnection on connection loss

SSH Worker Integration

- **Remote Execution:** Execute translation commands on remote workers
- **Connection Management:** Secure SSH connection handling
- **Progress Tracking:** Real-time progress from remote workers
- **Error Handling:** Comprehensive error detection and fallback

LLM Provider Support

- **OpenAI:** GPT-3.5, GPT-4, GPT-4 Turbo
- **Anthropic:** Claude-3 Opus, Claude-3 Sonnet, Claude-3 Haiku
- **DeepSeek:** DeepSeek Chat, DeepSeek Coder
- **Zhipu:** GLM-4, GLM-3 Turbo
- **Qwen:** Qwen Max, Qwen Plus, Qwen Turbo
- **Gemini:** Gemini Pro, Gemini Pro Vision
- **Ollama:** Local LLM support
- **LlamaCpp:** Local model execution

Dashboard Features

- **Progress Visualization:** Real-time progress bars and charts
- **Event Logging:** Comprehensive event history with filtering
- **Session Management:** Monitor multiple translation sessions
- **Worker Monitoring:** SSH worker status and performance
- **Responsive Design:** Works on desktop and mobile devices

Configuration

Environment Variables

```
# WebSocket Server
MONITOR_SERVER_PORT=8090

# SSH Workers
SSH_WORKER_HOST=localhost
SSH_WORKER_USER=milosvasic
SSH_WORKER_PASSWORD=your_password
SSH_WORKER_PORT=22
SSH_WORKER_REMOTE_DIR=/tmp/translate-ssh

# LLM APIs
OPENAI_API_KEY=your_openai_key
ANTHROPIC_API_KEY=your_anthropic_key
DEEPSEEK_API_KEY=your_deepseek_key
ZHIPU_API_KEY=your_zhipu_key
QWEN_API_KEY=your_qwen_key
GEMINI_API_KEY=your_gemini_key

# Logging
LOG_LEVEL=info # debug, info, warn, error
```

SSH Worker Configuration

```
{
  "distributed": {
    "enabled": true,
    "workers": {
      "thinker-worker": {
        "name": "Local Llama.cpp Worker",
        "host": "localhost",
        "port": 8444,
        "user": "milosvasic",
        "password": "password",
        "max_capacity": 10,
        "enabled": true,
        "tags": ["gpu", "llamacpp"]
      }
    },
    "ssh_timeout": 30,
    "ssh_max_retries": 3,
    "health_check_interval": 30
  }
}
```

Testing

Unit Tests

```
# Test WebSocket server
go test ./cmd/monitor-server

# Test event system
go test ./pkg/events

# Test SSH workers
go test ./pkg/sshworker

# Run all tests
go test ./...
```

Integration Tests

```
# Test WebSocket monitoring workflow
go run demo-websocket-client.go

# Test SSH worker integration
go run demo-ssh-worker-with-monitoring.go

# Test LLM integration
go run demo-real-llm-with-monitoring.go
```

```
# Comprehensive test
go run demo-comprehensive-monitoring.go
```

Performance Tests

```
# Benchmark WebSocket performance
go test -bench=. ./tests/websocket_monitoring_test.go

# Load test with multiple clients
for i in {1..10}; do
  go run demo-translation-with-monitoring-fixed.go &
done
```



Performance

WebSocket Performance

- **Connection Speed:** < 50ms to establish connection
- **Event Latency:** < 10ms per event
- **Throughput:** 100+ events/second
- **Concurrent Clients:** 50+ simultaneous connections

SSH Worker Performance

- **Connection Time:** < 5 seconds (configurable)
- **Command Execution:** < 1 second per translation
- **Retry Logic:** 3 automatic retries with exponential backoff
- **Connection Pooling:** Reuse connections for multiple commands

LLM Translation Performance

- **API Response:** 1-10 seconds depending on model
- **Token Limits:** 2000 tokens default (configurable)
- **Retry Logic:** Automatic retry on rate limits
- **Fallback:** Demo translation when API unavailable



Security

WebSocket Security

- **CORS Support:** Configurable origin restrictions
- **Rate Limiting:** 10 requests/second (configurable)
- **Authentication:** API key header support (optional)
- **Secure Mode:** WSS support for production

SSH Security

- **Key Authentication:** SSH key support recommended
- **Password Protection:** Secure password handling
- **Connection Limits:** Configurable timeout and retry limits
- **Command Restrictions:** Limited command execution scope

API Security

- **Key Management:** Secure API key storage
- **HTTPS Support:** TLS termination for API calls
- **Rate Limiting:** Provider-specific rate limit handling
- **Input Validation:** Comprehensive input sanitization



Deployment

Development

```
# Start monitoring server
go run ./cmd/monitor-server

# Run translation with monitoring
go run demo-comprehensive-monitoring.go

# Open dashboard
open enhanced-monitor.html
```

Production

```
# Build monitoring server
go build -o monitor-server ./cmd/monitor-server

# Build translation CLI
go build -o translator ./cmd/translator

# Start monitoring server
./monitor-server -config=production.json

# Run translation with monitoring
./translator -monitor -config=production.json input.txt output.md
```

Docker Deployment

```
FROM golang:1.21-alpine AS builder
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN go build -o monitor-server ./cmd/monitor-server
```

```
RUN go build -o translator ./cmd/translator
```

```
FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /app/monitor-server .
COPY --from=builder /app/translator .

EXPOSE 8090
CMD ["/monitor-server"]
```

Docker Compose

```
version: '3.8'
services:
  monitor-server:
    build: .
    ports:
      - "8090:8090"
    environment:
      - LOG_LEVEL=info
      - MONITOR_SERVER_PORT=8090
    volumes:
      - ./config.json:/app/config.json
      - ./logs:/app/logs

  translator:
    build: .
    depends_on:
      - monitor-server
    environment:
      - MONITOR_SERVER_URL=ws://monitor-server:8090/ws
    volumes:
      - ./input:/app/input
      - ./output:/app/output
```



Troubleshooting

Common Issues

- 1. WebSocket Connection Failed**
 - Check if monitoring server is running: `lsof -i :8090`
 - Start server: `go run ./cmd/monitor-server`
- 2. SSH Worker Connection Failed**
 - Test SSH connection: `ssh milosvasic@localhost`
 - Check credentials in config files
- 3. LLM API Authentication Failed**
 - Verify API key: `echo $OPENAI_API_KEY`
 - Set API key: `export OPENAI_API_KEY=your_key`
- 4. Port Conflicts**
 - Find process: `lsof -i :8090`
 - Kill process: `kill -9 <PID>`

- Use different port in config

Debug Mode

```
# Enable debug logging
export LOG_LEVEL=debug

# Run with verbose output
go run ./cmd/monitor-server -v -log-level=debug

# Monitor WebSocket traffic
websocat ws://localhost:8090/ws
```

Documentation

- **WebSocket Monitoring Guide** - Complete technical documentation
- **User Guide** - Step-by-step user instructions
- **Troubleshooting Guide** - Common issues and solutions
- **API Documentation** - REST and WebSocket API reference

Contributing

Development Setup

```
# Clone repository
git clone <repository-url>
cd translate

# Install dependencies
go mod tidy

# Run tests
go test ./...

# Start development server
go run ./cmd/monitor-server
```

Code Style

- Follow Go conventions and best practices
- Add comprehensive tests for new features
- Update documentation for API changes
- Use meaningful commit messages

Submitting Changes

1. Fork repository
2. Create feature branch

3. Add tests and documentation
4. Submit pull request

License

This project is licensed under the MIT License - see the [LICENSE](#) file for details.

Acknowledgments

- **Gorilla WebSocket** - WebSocket implementation
- **Chart.js** - Progress visualization charts
- **Tailwind CSS** - Dashboard styling
- **Go Community** - Various libraries and tools

Support

For support and questions: - Check documentation in docs/ directory - Review GitHub issues for known problems - Contact development team for assistance

Happy Monitoring! 